PIT Embedding Report Aug 21

PIT-1B Embedding Extraction: Hidden State Normalization

Summary

While extracting embeddings from PIT-1B for earnings call transcripts, we observed extremely large values in certain hidden dimensions (dim 1531 reaching magnitudes of 10,000+). After investigation, we found this is **not a model deficiency** but an **extraction error**: the model applies `F.rms_norm` to the hidden state before the LM head, and we were extracting the hidden state before this normalization step. Applying `F.rms_norm` to the extracted hidden states produces well-behaved embeddings across all 11 checkpoints (2014-2024).

Why This Happens

Hugging Face has built-in support for standard models like Llama-3, meaning it knows exactly where every piece is and how to handle it. When someone builds a custom model and uploads it to Hugging Face, they have to write their own instructions for how the model works internally, and if those instructions don’t expose every step the same way the standard models do, then tools that work automatically with Llama-3 will silently skip steps or return nothing when used on the custom model.

Root Cause

PIT is based on the modded-nanogpt architecture, which applies the final normalization as a bare functional call rather than a named module:

```
# PIT's forward method (modeling_pit.py):
for block in self.transformer["h"]:
    x = block(x, ...)
x = F.rms_norm(x, (x.size(-1),))   # ← functional call, not a named module
logits = self.lm_head(x)
```

Standard HuggingFace models (GPT-2, LLaMA) implement this as a named module (`self.norm` or `self.ln_f`) and return post-norm hidden states via `output_hidden_states=True`. PIT's implementation does not expose this because:

1. `F.rms_norm` is a bare function call with no learnable parameters, so it does not appear in `model.named_modules()` or `model.state_dict()`
2. `output_hidden_states=True` is not implemented in the custom `PITForCausalLM` (returns `None`, not pre-norm states)
3. The norm can be discovered by reading the `forward()` source code, tracing execution, or comparing captured hidden states against logits

This led us to extract the pre-normalization residual stream when using forward hooks on the last transformer block. The pre-norm values are not incorrect model output; they are the residual stream before the model's final normalization step, which may be useful for other analyses but are not the intended representation for downstream tasks.

Verification Across All Checkpoints

We tested the same 200 earnings call transcripts on all 11 PIT-1B checkpoints (2014-2024), extracting hidden states both with and without `F.rms_norm` applied after the last transformer block.

With `F.rms_norm` applied (post-normalization extraction)

Checkpoint	Std	Max	Dims > 10	L2 Norm
201412	0.90	32.2	2	32.7

Checkpoint	Std	Max	Dims > 10	L2 Norm
201511	0.90	35.0	2	34.1
201612	0.90	36.1	2	35.2
201712	0.90	36.7	1	35.9
201812	0.90	36.9	1	36.3
201912	0.90	36.8	2	36.3
202012	0.93	36.5	2	36.4
202112	0.93	35.4	2	36.3
202212	0.93	32.6	4	36.0
202312	0.89	27.8	5	35.4
202412	0.89	24.2	5	35.1

Stable across all checkpoints: std ~0.90, L2 norm ~35, at most 5 dimensions exceed abs 10. These values are numerically well-behaved. Downstream usefulness (e.g., predicting abnormal returns) requires task-specific validation.

Test setup: 200 earnings call transcripts (2014 Q1, first 2000 characters each, tokenized to max 512 tokens). Model inference in bfloat16, statistics computed in float32 after mean pooling.

Without `F.rms_norm` (pre-normalization residual stream)

Checkpoint	Std	Max	Dims > 10	L2 Norm	Top Dim Mean
201412	255	2,314	1,536	1,803	1,610
201511	255	4,160	1,536	3,090	2,929
201612	255	6,308	1,536	4,805	4,664
201712	255	8,924	1,536	6,975	6,825
201812	255	10,868	1,536	8,860	8,688
201912	255	12,797	1,536	10,211	9,980
202012	291	13,687	1,536	11,368	11,025
202112	299	13,549	1,536	11,667	11,048
202212	293	12,107	1,536	11,413	10,077
202312	268	9,469	1,536	10,600	7,907
202412	255	7,479	1,536	9,981	5,989

The magnitudes grow from 2014 to ~2020 (peaking around 202012-202112) then decrease. All 1,536 dimensions exceed abs 10. These values are entirely driven by the missing normalization step.

Why This Affects Researchers

The modded-nanogpt architecture (which PIT is based on) was designed for training speed, not for the HuggingFace ecosystem. The implementation gap:

Feature	Standard HuggingFace	PIT (modded-nanogpt)
Final norm	`self.norm = RMSNorm(...)` (named module)	`F.rms_norm(x, ...)` (functional call)
`output_hidden_states`	Returns post-norm states	Not implemented (returns None)
Hook on final norm	Works (`model.norm`)	Impossible (no module to hook)
`model.named_modules()`	Includes `norm`	Does not include any final norm

Researchers using forward hooks on the last transformer block will capture the pre-normalization residual stream and observe the large magnitude values. The `output_hidden_states=True` parameter returns `None` in the current implementation (not pre-norm states), so it fails silently rather than returning incorrect values.

PIT-4B Verification

The same pattern holds for the 4B model (4096 hidden dims, 20 layers). Tested on the same 200 transcripts with PIT-4B-201312:

	WITH rms_norm	WITHOUT rms_norm
Shape	(200, 4096)	(200, 4096)
Std	0.84	48.5
Max	17.9	1,361
Dims > 10	3	4,096 (100%)
L2 Norm	54.0	3,072

The 4B model's pre-norm magnitudes are smaller than 1B's (max 1,298 vs 13,687), consistent with reduced residual accumulation from 20 layers versus 52. The 4B's post-norm L2 norm (54.0) is naturally larger than 1B's (~36) due to the wider hidden dimension (4096 vs 1536), not a normalization issue. The same extraction correction (applying `F.rms_norm`) works for both model sizes.

Correct Extraction Method

```
import torch
import torch.nn.functional as F

# Hook on last transformer block
captured = {}
def hook_fn(module, input, output):
    captured["h"] = output[0] if isinstance(output, tuple) else
output

hook = model.transformer.h[-1].register_forward_hook(hook_fn)
model(input_ids=input_ids, attention_mask=attention_mask)
hook.remove()

# Apply the same rms_norm as the model's forward method
hidden = captured["h"]
hidden = F.rms_norm(hidden, (hidden.size(-1),))  # ← critical step

# Mean pool
mask = attention_mask.unsqueeze(-1).float()
embedding = (hidden * mask).sum(1) / mask.sum(1).clamp(min=1e-9)
```

Recommended Fix for the HuggingFace Repo

The cleanest solution is to add `output_hidden_states` support to `modeling_pit.py`. The conceptual change (to be adapted to the current file, which uses `CausalLMOutputWithPast` and supports caching):

```
# In forward(), add output_hidden_states parameter:
def forward(self, ..., output_hidden_states=None, **kwargs):
    if output_hidden_states is None:
        output_hidden_states = self.config.output_hidden_states

    all_hidden_states = [] if output_hidden_states else None
```

```
        for block in self.transformer["h"]:
            x = block(x, ...)
            if output_hidden_states:
                all_hidden_states.append(x)

        x = F.rms_norm(x, (x.size(-1),))

        if output_hidden_states:
            all_hidden_states.append(x)  # post-norm state

        # Preserve existing loss, past_key_values, logits_to_keep
behavior
        logits = self.lm_head(x)
        return CausalLMOutputWithPast(
            logits=logits,
            hidden_states=tuple(all_hidden_states) if
all_hidden_states else None,
            ...  # existing fields unchanged
        )
```

With this change, embedding extraction works out of the box:

```
    outputs = model(**inputs, output_hidden_states=True)
    embedding = outputs.hidden_states[-1]  # post-norm, 1536 dims,
ready to use
```

No hooks, no manual normalization. The `config.output_hidden_states` defaults to `False` so there is no memory overhead for standard text generation. When enabled, intermediate layer states and the final post-norm state are returned. Whether to also include the initial embedding state should be decided explicitly.

Note on Previous Report

The previous report (Aug 20) incorrectly identified an architectural issue. The model is architecturally sound. The issue was entirely in the extraction method.